

Developer



Back to All



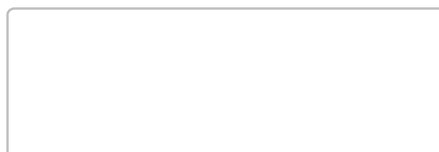
DEVELOPER BLOG

Improving memory bandwidth with the Tile Memory Heap extension for Vulkan



Written by
Patrick Boyle

May 27, 2026



Feedback



use memory and compute resources to keep pace.

With the introduction of Qualcomm Adreno High Performance Memory (HPM) with support for Tile Memory Heap, we're unlocking a new level of flexibility in the rendering pipeline—enabling developers to more intelligently manage on-chip memory, reduce bandwidth overhead, and optimize tile-based workflows. Enabling control over memory allocation and usage, these advancements empower developers to extract maximum performance from the GPU, paving the way for richer, more complex visual experiences without compromising efficiency or battery life.

Adreno HPM is high-speed memory on our GPU. A common use for this memory is [tile-based rendering](#) where the implementation divides the frame into tiles within tile memory and then transfers this data out to system memory. Using Adreno HPM can reduce memory bandwidth and power consumption by keeping memory contents local to the GPU which allows for more efficient memory access.

With the [VK_QCOM_tile_memory_heap](#) extension, a new use of tile memory becomes available. Applications can now bind high-traffic resources including images and buffers to tile memory for more efficient memory access across render and compute passes.

Optimizing memory access

Resources that are accessed multiple times across the application's frame are good candidates to be used with Adreno HPM. [VK_QCOM_tile_memory_heap](#) allows applications to make the decision on which resources reside in Adreno HPM which aims to

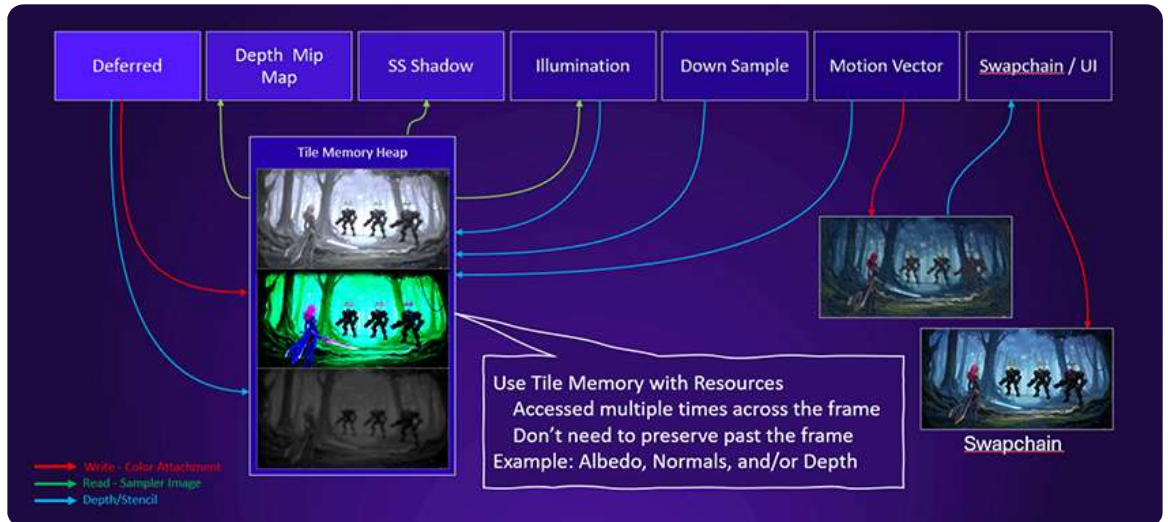


Sign up for Developer monthly newsletter

Join thousands of developers around the globe who receive latest news and updates from our monthly curated newsletter.

[Sign up >](#)

Let's look at a simple deferred rendering example:



Here the Albedo, Normals, and Depth Resources are first written in the Deferred Pass and then subsequently accessed in the following passes like the shadow and illumination pass. These 3 resources are good candidates for Adreno HPM because they are accessed multiple times in many render and compute passes across the frame.

Subheading – Tile memory properties

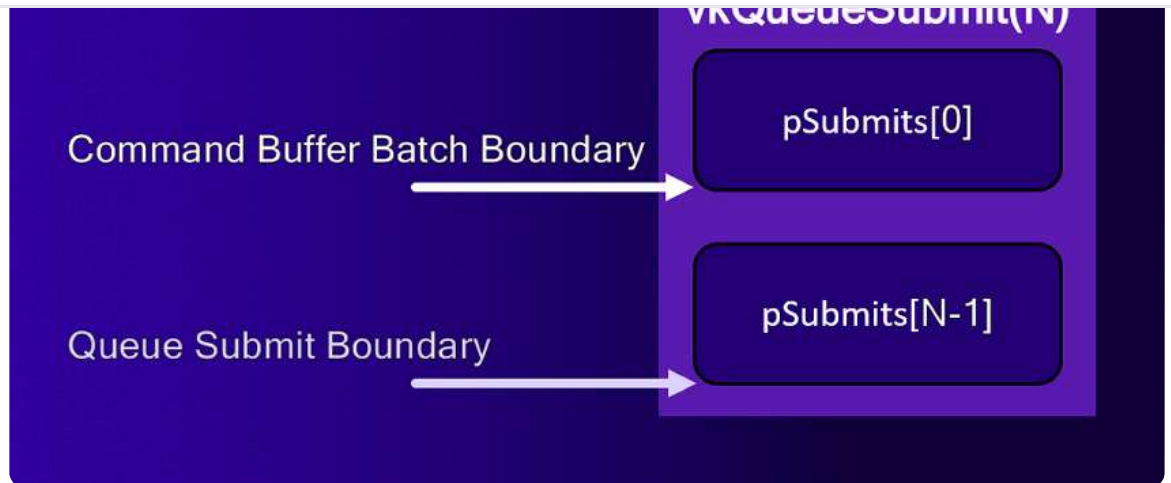
Our Adreno HPM is exposed via a new [VkMemoryHeap](#) on devices that support `VK_QCOM_tile_memory_heap`. Tile memory heaps will have the following property set [VK_MEMORY_HEAP_TILE_MEMORY_BIT_QCOM](#). The size of available tile memory can differ between devices and is typically small enough to only fit a small number of resources.



Come for support, stay for the community

Get support from experts, connect with like-minded developers, and access exclusive virtual events.

[Join Developer Discord](#) >



Adreno HPM is transient and the scope of where tile memory is defined is only within a [single command buffer submission batch](#) by default. On some devices this boundary extends further to a [queue submit](#) which can be checked with the new [queueSubmitBoundary](#) physical device property.

The picture above illustrates a Queue Submit with N # of Command Buffer Batches and where the defined boundary occurs based off if this physical device property is set.

Adreno HPM works best on resources that only have a limited lifetime within a frame of an application. All memory contents within tile memory eventually become undefined when not within the defined [tile memory scope](#).

If any contents from tile memory need to be preserved outside this scope then the application must issue a transfer command from a tile memory resource to a non-tile memory resource in order to keep a copy of the memory contents but this may alter performance so only use it when absolutely needed.

In the deferred rendering example above, all 3 deferred rendering resources are completely re-written every frame and the memory contents are no

This means that these resources are good candidates to be used with Adreno HPM as their memory contents can be discarded and don't need to be saved out to system memory in the frame.

Using Adreno HPM with resources

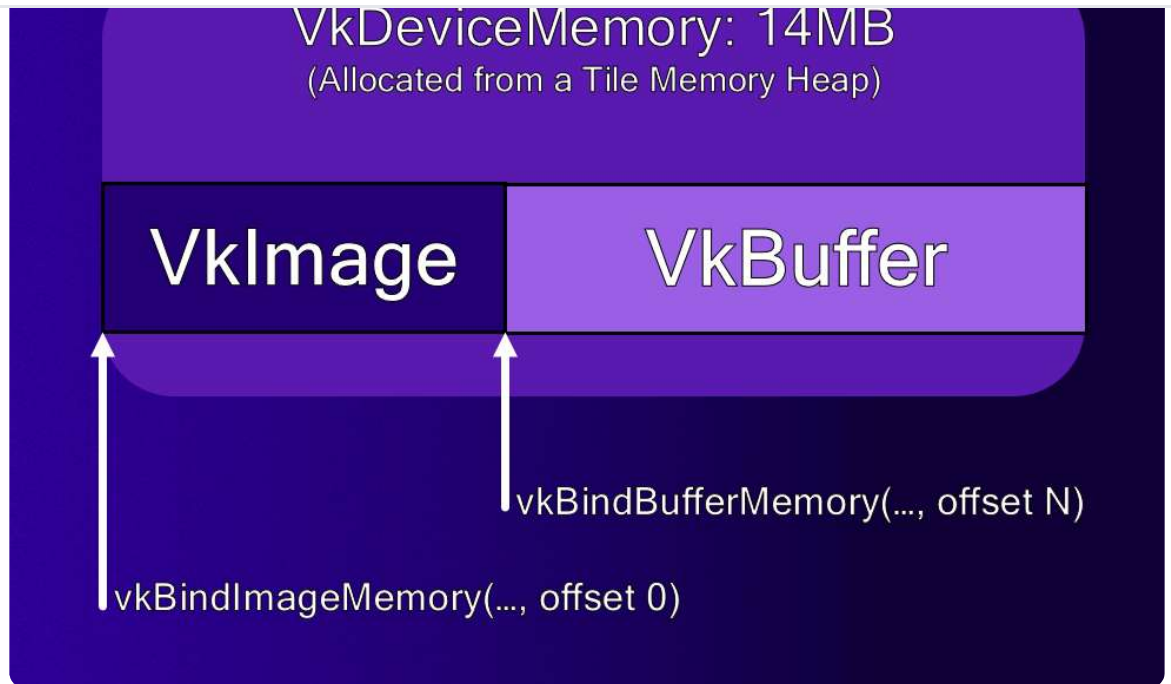
To bind our Adreno HPM to a resource, the following new usage flags must be added for image and buffer creation respectively

[VK_IMAGE_USAGE_TILE_MEMORY_BIT_QCOM](#) and [VK_BUFFER_USAGE_TILE_MEMORY_BIT_QCOM](#).

When querying memory requirements for these tile memory resources, the new structure [VkTileMemoryRequirementsQCOM](#) must be provided to the proper [vkGetImageMemoryRequirements2](#) or [vkGetBufferMemoryRequirements2](#) call.

Our Adreno driver will fill in the tile memory requirements for the given resource if it is supported in our Adreno HPM. If for any reason this resource is not supported in tile memory for whatever reason, like if the resource is too large to fit in the tile memory heap, the 'size' and 'alignment' fields of the [VkTileMemoryRequirementsQCOM](#) structure will be filled in with 0's when querying memory requirements.

Once the memory requirements have been gathered for the resources, these requirements must be used to bind a [VkDeviceMemory](#) object allocated from the tile memory heap to the resource. You can now bind tile memory to a resource like you would an existing [VkDeviceMemory](#) object to gain the benefits of our Adreno HPM.



In the picture above, `VkImage` and `VkBuffer` are both bound to the same tile memory `VkDeviceMemory` object. Both resources are able to fit within 14 Megabytes of tile memory simultaneously without any overlap. Overlapping of resources is allowed in tile memory and existing [memory aliasing](#) rules apply from the Vulkan spec.

Allocating a `VkDeviceMemory` object from the tile memory heap will **always** return a `VkDeviceMemory` object with an offset of 0 into the tile memory heap. Allocating tile memory does not consume any actual memory from this heap. This means that an application may have different `VkDeviceMemory` objects allocated out of a tile memory heap that represents different tile memory layouts.

Binding tile memory in command buffers

Binding tile memory to resources is only the first step of using tile memory with this extension. In order to access the tile memory during command execution, **tile memory must also be bound to the command buffer at the time of execution.**

memory will be reserved by the application. If any amount of tile memory from the heap is unbound, the implementation is free to use any unbound tile memory for other internal uses such as tile-based rendering.

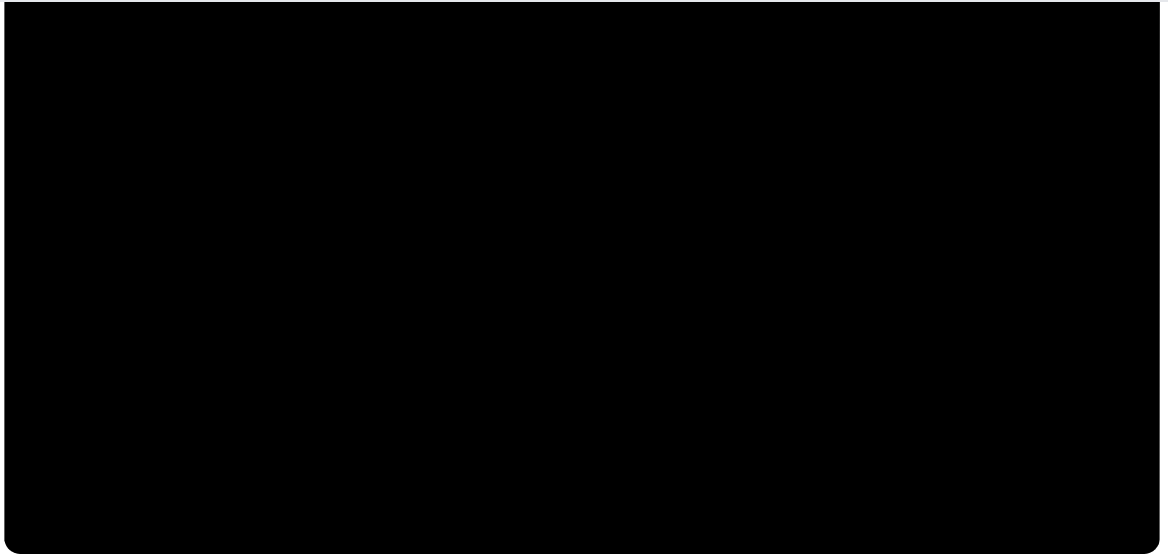
To bind tile memory to a command buffer a new API is provided [vkCmdBindTileMemoryQCOM](#) which will bind the provided [tile memory VkDeviceMemory](#) object to the command buffer. The bound tile memory object represents a range of tile memory that the application can access and persist across commands. This command can be called with a null pointer in order to bind no tile memory to the command buffer.

Tile memory is implicitly unbound at the end of a command buffer and must be rebound in the next command buffer if memory contents are expected to persist across command buffers within the tile memory scope.

Note that any `VkDeviceMemory` being bound to the command buffer must be the same `VkDeviceMemory` bound to the resources accessed in the executed commands.

Because Tile Memory `VkDeviceMemory` objects allocated always start at offset 0 into the heap, an application may create as many `VkDeviceMemory` layouts as they'd like with different resources aliasing different memory locations as needed. Allocating memory from the tile memory heap does not consume any of the total memory.

Tile memory in action



Here is an animation that shows what happens to the tile memory heap as commands get executed. This example shows 2 separate command buffers within a single Command Buffer batch which use the Tile Memory Heap.

While the commands get executed, only the bound tile memory range becomes defined and is reserved for the application. The red indicates undefined memory contents due to unbound tile memory.

Notice that the contents of tile memory get preserved between Command Buffer 0 and Command Buffer 1 even though the tile memory object is implicitly unbound after the last command finishes in Command Buffer 0.

Since Command Buffer 1 binds a separate `VkDeviceMemory B` before an action command is executed, the memory contents are preserved and can be accessed within Command Buffer 1.

When the last command in Command Buffer 1 completes, all tile memory is discarded and becomes undefined because Command Buffer 1 is the last Command Buffer in the submission batch tile memory scope.

to preserve memory contents from frame to frame and are accessed across different render and compute passes

- Use [memory aliasing](#) to fit multiple resources in the small amount of tile memory available. Memory aliasing can also allow different tile memory layouts to grow and shrink the bound tile memory within the command buffer
- Bind only the range of tile memory that is needed to the command buffer to allow the implementation to use any unbound tile memory for internal optimizations
- Using Vulkan memory allocation libraries like [VMA](#) can help with sub allocating resources
- Use [Snapdragon Profiler](#) to measure read/write bandwidth changes when using resources in tile memory
- For more details and interactions with other tile QCOM extensions check out the [proposal](#)
- Code sample:
https://github.com/SnapdragonGameStudios/adr-eno-gpu-vulkan-code-sample-framework/tree/main/samples/tile_memory
- Validation Layer support in SDK 1.4.341.0

Have questions? Get in touch with our team



Opinions expressed in the content posted here are the personal opinions of the original authors, and do not necessarily reflect those of Qualcomm Incorporated or its subsidiaries ("Qualcomm"). The content is provided for informational purposes only and is not meant to be an endorsement or representation by Qualcomm or any other party. This site may also provide links or references to non-Qualcomm sites and resources. Qualcomm makes no representations, warranties, or other commitments whatsoever about any non-Qualcomm sites or third-party resources that may be referenced, accessible from, or linked to this site.

Snapdragon and Qualcomm branded products are products of Qualcomm Technologies, Inc. and/or its subsidiaries.

About the Author



Patrick Boyle
Engineer, Staff at
Qualcomm
Technologies, Inc.

[Read More](#)
[About this author +](#)

Related Articles



Feedback





code

J

Jul 7, 2026

Open Source

On-Device AI Generative AI NPU



Qualcomm relentlessly innovates to deliver intelligent computing everywhere, helping the world tackle some of its most important challenges. Our leading-edge AI, high performance, low-power computing, and unrivaled connectivity deliver proven solutions that transform major industries. At Qualcomm, we are engineering human progress.



Quick links

- Products
- Support
- Partners
- Contact us
- Developer

Company info

- About us
- Careers
- Investors
- News & media
- Our businesses
- Email Subscriptions

Stay connected

Get the latest Qualcomm and industry information delivered to your inbox.



Manage your subscription

Feedback





Snapdragon 8 Gen 3 Mobile Platform

- Terms of Use
- Privacy
- Cookie Policy
- Accessibility Statement
- Responsible AI Policy
- Cookie Settings
- Language: English (US)

© Qualcomm Technologies, Inc. and/or its affiliated companies.

Snapdragon and Qualcomm branded products are products of Qualcomm Technologies, Inc. and/or its subsidiaries. Qualcomm patented technologies are licensed by Qualcomm Incorporated.

Note: Certain services and materials may require you to accept additional terms and conditions before accessing or using those items.

References to "Qualcomm" may mean Qualcomm Incorporated, or subsidiaries or business units within the Qualcomm corporate structure, as applicable.

Qualcomm Incorporated includes our licensing business, QTL, and the vast majority of our patent portfolio. Qualcomm Technologies, Inc., a subsidiary of Qualcomm Incorporated, operates, along with its subsidiaries, substantially all of our engineering, research and development functions, and substantially all of our products and services businesses, including our QCT semiconductor business.

Materials that are as of a specific date, including but not limited to press releases, presentations, blog posts and webcasts, may have been superseded by subsequent events or disclosures.

Nothing in these materials is an offer to sell or license any of the services or materials referenced herein.

